Fibonacci with Dynamic Programming

The Fibonacci sequence is a classic intro to DP. It is deceptively simple but teaches the core idea: don't recompute what you already know.

Definition:

$Fib(0) = 0$
$Fib(1) = 1$
$Fib(n) = Fib(n-1) + Fib(n-2)$ for $n \geq 2$

Here are the first 7 values:

0	1	1	2	3	5	8
0	1	2	3	4	5	6

The Recurrence - Why It Works

Each number is the sum of the two before it. That is it! The recurrence captures a simple „build up from smaller answers" structure.

$Fib(2) = Fib(1) + Fib(0) = 1 + 0 = 1$
$Fib(3) = Fib(2) + Fib(1) = 1 + 1 = 2$
$Fib(4) = Fib(3) + Fib(2) = 2 + 1 = 3$

Naive recursion re-solves the same subproblems over and over - $Fib(2)$ gets computed multiple times in a recursive tree. This blows up to $O(2^n)$ time!

DP fixes this by storing results in a table so each subproblem is solved once.

Building the DP Table

We fill the table bottom-up, starting from the base cases:

n	dp[n]	computation
0	0	base case
1	1	base case
2	1	1+0
3	2	1+1
4	3	2+1
5	5	3+2
6	8	5+3

Notice each row only looks at the two rows above it. That is the key to the space optimization below.

Pseudocode (bottom-up):

```
dp[0] = 0
dp[1] = 1
for i = 2 to n:
    dp[i] = dp[i-1] + dp[i-2]
return dp[n]
```

Complexity Analysis

Time: $O(n)$ - we compute $n-1$ values, each in $O(1)$.
Space: $O(n)$ for the full array.

But we only ever need the last two values! So we can drop the array and use two variables:

```
prev2 = 0  // Fib(0)
prev1 = 1  // Fib(1)
for i = 2 to n:
    curr = prev1 + prev2
    prev2 = prev1
    prev1 = curr
```

This brings space down to O(1). Same O(n) time, constant memory.

Key Takeaways

DP replaces recursion by storing subproblem results - trading a bit of me